Software Design and Construction - SWE 316

Assignment 1

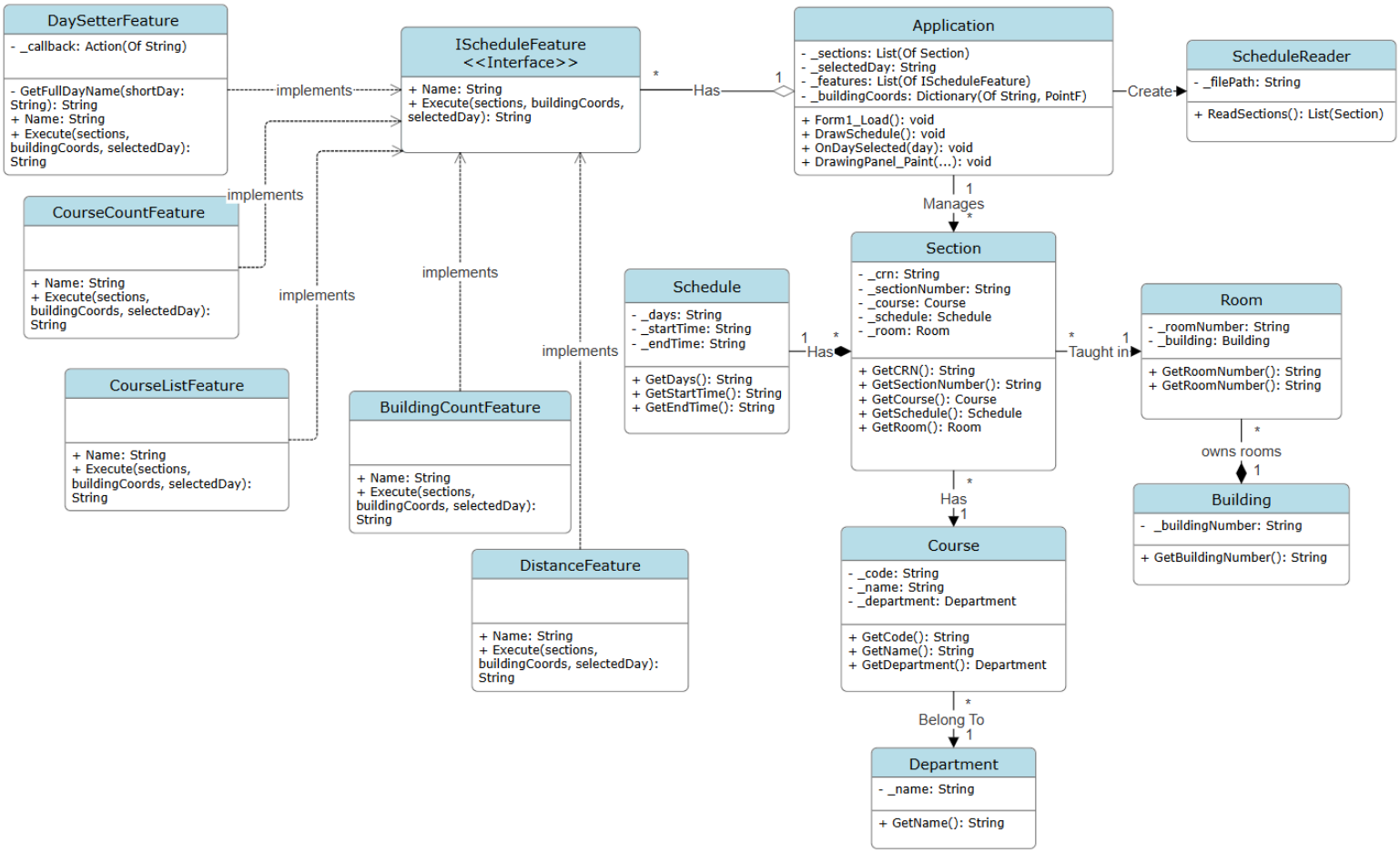
25-10-2025

Raneem Alshahrani

202277080

Task	Grade	Your Grade	Comments
Task # 1: Class Diagram	15		
Task # 2: Application	55		
Check list and penalties No Cover page with grade table -5 ☐ File name (report) -5 ☐ Not in PDF format -5 ☐			
Total	70		

Class Diagram:



Code implementation:

1- ScheduleReader class:

The ScheduleReader class is reading data from Excel then saving it into objects and loading the data into Application.

```
Imports System.Globalization
Imports Microsoft.Office.Interop

Public Class ScheduleReader
    Private _filePath As String

    Public Sub New(filePath As String)
        _filePath = filePath
    End Sub

    Public Function ReadSections() As List(Of Section)
        Dim sections As New List(Of Section)()
        Dim excelApp As New Excel.Application()
        Dim workbook As Excel.Workbook = Nothing
        Dim worksheet As Excel.Worksheet = Nothing

        Try
            workbook = excelApp.Workbooks.Open(_filePath)
            worksheet = CType(workbook.Worksheets(1), Excel.Worksheet) ' First sheet
            Dim usedRange As Excel.Range = worksheet.UsedRange
            Dim rowCount As Integer = usedRange.Rows.Count

            For row As Integer = 2 To rowCount ' Skip header row
                Dim crn As String = If(worksheet.Cells(row, 2).Value, "").ToString()
                Dim sectionNumber As String = If(worksheet.Cells(row, 5).Value, "").ToString()
                Dim courseCode As String = If(worksheet.Cells(row, 3).Value, "").ToString()
                Dim courseName As String = If(worksheet.Cells(row, 6).Value, "").ToString()
                Dim departmentName As String = If(worksheet.Cells(row, 4).Value, "").ToString()
                Dim days As String = If(worksheet.Cells(row, 8).Value, "").ToString()
                Dim startTime As String = If(worksheet.Cells(row, 9).Value, "").ToString()
                Dim endTime As String = If(worksheet.Cells(row, 10).Value, "").ToString()
                Dim buildingNumber As String = If(worksheet.Cells(row, 11).Value, "").ToString()
                Dim roomNumber As String = If(worksheet.Cells(row, 12).Value, "").ToString()

                ' Create objects
                Dim department As New Department(departmentName)
                Dim course As New Course(courseCode, courseName, department)
                Dim building As New Building(buildingNumber)
                Dim room As New Room(roomNumber, building)
                Dim schedule As New Schedule(days, startTime, endTime)
                Dim section As New Section(crn, sectionNumber, course, schedule, room)
                sections.Add(section)
            Next
        Catch ex As Exception
            MessageBox.Show("Error reading file: " & ex.StackTrace)
        Finally
            If workbook IsNot Nothing Then workbook.Close(False)
            excelApp.Quit()
        End Try

        Return sections
    End Function
End Class
```

Stores file path.

Opens Excel workbook and selects the first worksheet.

Iterates rows (2 rowCount), reads cell values safely. Instantiates Department, Course, Building, Room, and Schedule objects.

Creates Section objects and adds them to the sections list.

Closes Excel resources.

Return the list.

2- Section class:

The Section class provides course section that takes place in a certain room and schedule.

```
The Section class
Public Class Section
    ' Private fields
    Private _crn As String
    Private _sectionNumber As String

    Private _course As Course
    Private _schedule As Schedule
    Private _room As Room

    ' Constructor
    Public Sub New(crn As String, sectionNumber As String, course As Course, schedule As Schedule, room As
Room)
        _crn = crn
        _sectionNumber = sectionNumber
        _course = course
        _schedule = schedule
        _room = room
    End Sub

    ' Getter methods
    Public Function GetCRN() As String
        Return _crn
    End Function
    Public Function GetSectionNumber() As String
        Return _sectionNumber
    End Function
    Public Function GetCourse() As Course
        Return _course
    End Function
    Public Function GetSchedule() As Schedule
        Return _schedule
    End Function
    Public Function GetRoom() As Room
        Return _room
    End Function

    ' ToString method for debugging
    Public Overrides Function ToString() As String
        Return "Section " & _sectionNumber & " (CRN: " & _crn & ") for " & _course.GetCode() & " in " &
_room.GetRoomNumber()
    End Function
End Class
```

Stores CRN and Section Number for Section.

References to Course, Schedule, and Room.

Constructor for Section class.

Provides getters for CRN, Section Number, Course, Schedule, and Room.

ToString summarizes section details for debugging.

3- Course class:

The Course class provides university Course information such as, code, name, and department.

```
Public Class Course
    ' Private fields
    Private _code As String
    Private _name As String

    Private _department As Department

    ' Constructor
    Public Sub New(code As String, name As String, department As Department)
        _code = code
        _name = name
        _department = department
    End Sub

    ' Getter methods
    Public Function GetCode() As String
        Return _code
    End Function
    Public Function GetName() As String
        Return _name
    End Function
    Public Function GetDepartment() As Department
        Return _department
    End Function

    ' ToString method for debugging
    Public Overrides Function ToString() As String
        Return _code & " - " & _name & " (" & _department.GetName() & ")"
    End Function
End Class
```

Stores code and name for Section.

References to Department.

Constructor for Course class.

Provides getters for code, Name, Department for Section.

ToString formats a readable label.

4- Department class:

The Department class provides department names.

```
Public Class Department
    ' Private field
    Private _name As String

    ' Constructor
    Public Sub New(name As String)
        _name = name
    End Sub

    ' Getter method
    Public Function GetName() As String
        Return _name
    End Function

    ' ToString method for debugging
    Public Overrides Function ToString() As String
        Return "Department: " & _name
    End Function
End Class
```

Stores name for Department.

Constructor for Department class.

Provides getters for Name for Department.

ToString formats a readable label.

5- Room class:

The Room class provides room number and building.

```
Public Class Room
    ' Private attributes
    Private _roomNumber As String

    Private _building As Building

    ' Constructor
    Public Sub New(roomNumber As String, building As Building)
        _roomNumber = roomNumber
        _building = building
    End Sub

    ' Public getter methods
    Public Function GetRoomNumber() As String
        Return _roomNumber
    End Function

    Public Function GetBuilding() As Building
        Return _building
    End Function

    ' ToString method for debugging
    Public Overrides Function ToString() As String
        Return "Room " & _roomNumber & " in " & _building.GetBuildingNumber()
    End Function
End Class
```

Stores number for Room.

References to Building.

Constructor for Room class.

Provides getters for Room number and Building.

ToString formats a readable label.

6- Building class:

The Building class provides building number.

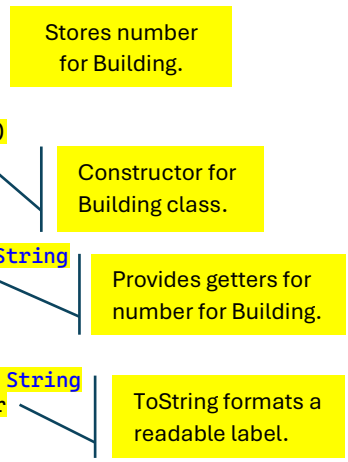
```
Public Class Building

    ' Private field
    Private _buildingNumber As String

    ' Constructor
    Public Sub New(buildingNumber As String)
        _buildingNumber = buildingNumber
    End Sub

    ' Getter method
    Public Function GetBuildingNumber() As String
        Return _buildingNumber
    End Function

    ' ToString method for better debugging
    Public Overrides Function ToString() As String
        Return "Building " & _buildingNumber
    End Function
End Class
```



7- Schedule class:

The Schedule class provides specific day and starts time and end time for section.

```
Public Class Schedule
    ' Private fields
    Private _days As String
    Private _startTime As String
    Private _endTime As String

    ' Constructor
    Public Sub New(days As String, startTime As String, endTime As String)
        _days = days
        _startTime = startTime
        _endTime = endTime
    End Sub

    ' Getter methods
    Public Function GetDays() As String
        Return _days
    End Function

    Public Function GetStartTime() As String
        Return _startTime
    End Function

    Public Function GetEndTime() As String
        Return _endTime
    End Function

    ' ToString method for debugging
    Public Overrides Function ToString() As String
        Return "Schedule: " & _days & " from " & _startTime & " to " & _endTime
    End Function
End Class
```

Stores days, start time, and end time for schedule.

Constructor for Schedule class.

Provides getters for days, start time, and end time for schedule.

ToString formats a readable label.

8- IScheduleFeature interface:

The IScheduleFeature interface is used to apply Open-Closed Principle for the features in textbox.

```
' This interface defines a contract for all schedule-related features.
' Each implementing class must provide a unique name and an Execute method
' that performs a specific operation on the provided schedule data.
Public Interface IScheduleFeature

    ' A human-readable name identifying the feature.
    ' Used for display, logging, or feature selection.
    ReadOnly Property Name As String

    ' The core method that every feature must implement.
    ' Parameters:
    '   sections      - A list of Section objects representing the schedule.
    '   buildingCoords - A dictionary mapping building identifiers to their coordinates.
    '   selectedDay    - The currently selected day for context-sensitive features.
    ' Returns:
    '   A string describing the result or output of the feature.
    Function Execute(sections As List(Of Section),
                     buildingCoords As Dictionary(Of String, PointF),
                     selectedDay As String) As String
End Interface
```

Name property for feature modules.

Execute receives sections list, building coordinates, and selected day.

9- DaySetterFeature class:

The DaySetterFeature class provides full day name of selected day button for given CRNs.

```
' This class implements the IScheduleFeature interface to allow
' setting or updating the currently selected day in a schedule system.
Public Class DaySetterFeature
    Implements IScheduleFeature

    ' A private callback delegate that will be invoked when the day is set.
    Private ReadOnly _callback As Action(Of String)

    ' Constructor that takes a callback function.
    ' The callback allows external code to react whenever a new day is selected.
    Public Sub New(callback As Action(Of String))
        _callback = callback
    End Sub

    ' Property representing the name of this feature.
    ' Returns a constant string identifier for clarity.
    Public ReadOnly Property Name As String Implements IScheduleFeature.Name
        Get
            Return "Day Setter"
        End Get
    End Property

    ' Executes the feature's logic:
    ' - Invokes the callback with the selected day code.
    ' - Converts the short day code into a full day name.
    ' - Returns a formatted string showing the selected day.
    Public Function Execute(sections As List(Of Section),
                            buildingCoords As Dictionary(Of String, PointF),
                            selectedDay As String) As String Implements IScheduleFeature.Execute

        _callback(selectedDay)
        Dim fullDay = GetFullDayName(selectedDay)
        Return $"Selected Day: {fullDay}"

    End Function

    ' Helper function that maps short day codes to their full names.
    ' If an unrecognized code is passed, it simply returns it unchanged.
    Private Function GetFullDayName(shortDay As String) As String
        Select Case shortDay
            Case "U" : Return "Sunday"
            Case "M" : Return "Monday"
            Case "T" : Return "Tuesday"
            Case "W" : Return "Wednesday"
            Case "R" : Return "Thursday"
            Case Else : Return shortDay
        End Select
    End Function
End Class
```

Implements IScheduleFeature.

sets selected day via callback.

Constructor for DaySetterFeature class.

Property representing the name of this feature and returns constant string

Execute takes list of sections, coordination, and selected day.

Converts short day code to full .

Function to return full name of the day.

10- CourseCountFeature class:

The CourseCountFeature class provides number of courses for given CRNs.

```
' This class implements the IScheduleFeature interface to provide
' functionality for counting the total number of courses in the given schedule.
Public Class CourseCountFeature
    Implements IScheduleFeature

    ' Property representing the name of this feature.
    ' Returns a fixed string that identifies the feature.
    Public ReadOnly Property Name As String Implements IScheduleFeature.Name
        Get
            Return "Course Counter"
        End Get
    End Property

    ' Executes the feature logic:
    ' Accepts a list of sections, a dictionary mapping building IDs to coordinates,
    ' and a selected day (unused here).
    ' Returns a formatted string showing how many courses are in the list.
    Public Function Execute(sections As List(Of Section),
                            buildingCoords As Dictionary(Of String, PointF),
                            selectedDay As String) As String Implements IScheduleFeature.Execute
        Return $"Number of Courses = {sections.Count}"
    End Function
End Class
```

Implements IScheduleFeature.

Property representing the name of this feature and returns constant string

Use Execute to take CRNs needed information.

Return the number of courses.

11- CourseListFeature class:

The CourseListFeature class provides lists of courses for given CRNs.

```
' This class implements the IScheduleFeature interface
' to list all courses in the provided schedule.
Public Class CourseListFeature
    Implements IScheduleFeature

    ' Property representing the name of the feature.
    ' Returns a fixed descriptive string.
    Public ReadOnly Property Name As String Implements IScheduleFeature.Name
        Get
            Return "Course Lister"
        End Get
    End Property

    ' Executes the feature logic:
    ' Takes a list of sections, a dictionary mapping building IDs to coordinates,
    ' and a selected day (not used here).
    ' Returns a formatted list of all courses with their codes and names.
    Public Function Execute(sections As List(Of Section),
                            buildingCoords As Dictionary(Of String, PointF),
                            selectedDay As String) As String Implements IScheduleFeature.Execute

        ' StringBuilder for efficient string concatenation.
        Dim result As New Text.StringBuilder()

        ' Index counter to number each listed course.
        Dim idx As Integer = 1

        ' Iterate through all sections and extract course info.
        For Each s In sections
            result.AppendLine($"{idx}- {s.GetCourse().GetCode()}: {s.GetCourse().GetName()}")
            idx += 1
        Next

        ' Return the final formatted course list, trimmed of extra line breaks.
        Return result.ToString().Trim()
    End Function
End Class
```

Implements IScheduleFeature.

Property representing the name of this feature and returns constant string

Use Execute to take CRNs needed information.

Loop for building list for courses.

Return lists for courses.

12- BuildingCountFeature class:

The BuildingCountFeature class provides counting the number of distinct Building for given CRNs.

```
' This class implements the IScheduleFeature interface to provide
' functionality for counting distinct buildings in a schedule.
Public Class BuildingCountFeature
    Implements IScheduleFeature

    ' Property representing the name of the feature.
    ' Returns a constant descriptive string.
    Public ReadOnly Property Name As String Implements IScheduleFeature.Name
        Get
            Return "Building Counter"
        End Get
    End Property

    ' Executes the feature's logic:
    ' Takes a list of sections, a dictionary mapping building IDs to coordinates,
    ' and the selected day as input, then counts the number of distinct buildings.
    Public Function Execute(sections As List(Of Section),
                            buildingCoords As Dictionary(Of String, PointF),
                            selectedDay As String) As String Implements IScheduleFeature.Execute

        ' Extracts building numbers from the sections,
        ' filters out duplicates, and counts how many unique buildings exist.
        Dim uniqueBuildings = sections.
            Select(Function(s) s.GetRoom().GetBuilding().GetBuildingNumber()).
            Distinct().
            Count()

        ' Returns the result as a formatted string.
        Return $"Number of Different Buildings = {uniqueBuildings}"
    End Function
End Class
```

Implements IScheduleFeature.

Property representing the name of this feature and returns constant string

Use Execute to take CRNs needed information.

Count distinct building.

Return the number of distinct building.

13- DistanceFeature class:

The DistanceFeature class provides total distance for given CRNs.

```
' This class implements the IScheduleFeature interface to calculate
' the total travel distance between consecutive buildings in a schedule.
Public Class DistanceFeature
    Implements IScheduleFeature

    ' Property representing the name of the feature.
    ' Returns a descriptive string for identification.
    Public ReadOnly Property Name As String Implements IScheduleFeature.Name
        Get
            Return "Distance Calculator"
        End Get
    End Property

    ' Executes the feature logic:
    ' Takes a list of sections, building coordinates, and the selected day (unused here).
    ' Calculates the total Euclidean distance between consecutive buildings in the schedule.
    Public Function Execute(sections As List(Of Section),
                            buildingCoords As Dictionary(Of String, PointF),
                            selectedDay As String) As String Implements IScheduleFeature.Execute

        ' Accumulator for total distance.
        Dim totalDist As Double = 0

        ' Iterate through all adjacent section pairs.
        For i = 0 To sections.Count - 2
            ' Get building numbers for the current and next section.
            Dim b1 = sections(i).GetRoom().GetBuilding().GetBuildingNumber()
            Dim b2 = sections(i + 1).GetRoom().GetBuilding().GetBuildingNumber()

            ' Only calculate distance if both buildings have known coordinates.
            If buildingCoords.ContainsKey(b1) AndAlso buildingCoords.ContainsKey(b2) Then
                Dim p1 = buildingCoords(b1)
                Dim p2 = buildingCoords(b2)

                ' Use the Euclidean distance formula: √((x1 - x2)² + (y1 - y2)²)
                totalDist += Math.Sqrt((p1.X - p2.X) ^ 2 + (p1.Y - p2.Y) ^ 2)
            End If
        Next

        ' Return the total distance, rounded to two decimal places.
        Return $"Distance Traveled = {Math.Round(totalDist, 2)} m"
    End Function
End Class
```

Implements IScheduleFeature.

Property representing the name of this feature and returns constant string

Use Execute to take CRNs needed information.

Initiate totalDist.

Take two building coordinates.

Check if the building has known coordinates.

Use the Euclidean distance formula to find total distance.

Find totalDist then round it into 2 decimals.